05. On-Campus Interview Preparation Guide & Defense

1. The 30-Second Elevator Pitch

> *"We built Internship Tracker, an automated early-careers intelligence engine tailored for Computer Science students in India. Unlike generic job boards that are flooded with dead links, staffing agencies, and unpaid roles, our platform automatically ingests, normalizes, and verifies requisitions directly from enterprise ATS platforms like Greenhouse, Lever, Ashby, and Workday. It strictly enforces a ₹30,000/month stipend floor, standard 2–3 month summer durations, and guarantees 100% active, direct application URLs using live automated HTTP verification. It runs as a Next.js 14, Express, and PostgreSQL monorepo with an automated self-healing background crawler."*

---

2. The 2-Minute In-Depth Walkthrough (STAR Method)

Situation (S)

During university internship drives, thousands of engineering students struggle to find verified, direct openings at top tech firms. Most job aggregators scrape stale career portals, leading to broken 404 links, generic landing pages, or misleading 6-month co-ops that conflict with college academic calendars.

Task (T)

Our goal was to architect a high-reliability, full-stack platform that discovers active technical internships directly from employer ATS platforms, cleans and classifies the data, verifies every application link with live network health checks, and presents the listings via a high-performance web application.

Action (A)

1. **Multi-Source Ingestion Pipeline**: Designed native connectors querying Greenhouse, Lever, Ashby, and Workday CXS APIs across 200+ tier-1 tech companies in India.
2. **Data Normalization & Classification**: Built a regex and NLP-based engine that classifies roles into SDE, AI/ML, Data, and Core Tech, while extracting structured stipends, durations, and skills.
3. **Deterministic Deduplication**: Engineered a SHA-256 fingerprinting algorithm based on the employer slug, normalized title, and unique requisition token, backed by a PostgreSQL unique index.
4. **Live Link Health Verifier**: Implemented an automated HTTP GET verifier with realistic browser headers that tests links against corporate WAFs (Cloudflare/Fastly) and automatically marks dead requisitions as `CLOSED`.
5. **Modern Full-Stack Architecture**: Built the frontend using Next.js 14 App Router for fast server-side rendering and responsive Tailwind CSS UI, backed by an Express REST API and a PostgreSQL database on Neon Cloud with Prisma ORM.

Result (R)

- Built a production-ready directory with 50+ live-verified, high-stipend (₹35k to ₹1.8L/month) student internships across 30 top-tier tech companies.
- Achieved **100% link accuracy** (zero broken links) through automated verification.
- Sub-50ms API query latency through composite database indexing and gzip payload compression.

---

3. Top 20 Technical Interview Questions & Answers

System Design & Ingestion

Q1: Why did you scrape ATS APIs instead of scraping career page HTML with Puppeteer/Selenium?

Answer:
* **Reliability**: HTML DOM scrapers break whenever a company redesigns its frontend CSS or DOM hierarchy. ATS APIs (Greenhouse, Lever, Ashby, Workday CXS) return structured JSON that adheres to a stable schema.
* **Performance & Resource Consumption**: Headless browser automation (Puppeteer/Playwright) consumes hundreds of megabytes of RAM per browser instance and requires significant CPU cycles to execute JavaScript. By using lightweight HTTP REST requests (`fetch`), our ingestion pipeline can audit 100+ company boards in seconds with minimal memory overhead.

Q2: How do you handle anti-bot Web Application Firewalls (WAFs) like Cloudflare or Fastly?

Answer:
* Many corporate job boards block default HTTP clients like `curl` or Node.js default User-Agents (`node-fetch`) with HTTP 403 Forbidden.
* We configure full modern browser headers (`User-Agent` mimicking macOS Chrome, `Accept: text/html,application/xhtml+xml` , `Accept-Language` , and `Cache-Control`).
* We also configure realistic request timeouts (8s with a 12s retry) and staggered parallel batching (chunks of 10–25 requests) to avoid triggering IP-level rate limit thresholds.

Q3: How does the system detect when an internship has expired or closed?

Answer:
* Our automated background scheduler runs every 30 minutes.
* During each sweep, it queries the employer's ATS. If a requisition was previously listed as `OPEN` in our database but is no longer returned in the active ATS feed, or if its direct application URL returns HTTP 404 / 410, the pipeline immediately updates its database status to `CLOSED` .
* This ensures students never waste time applying to expired openings.

Database, Deduplication & Concurrency

Q4: How do you solve the duplicate listing problem?

Answer:
* Employers often list the same position across multiple cities or update titles slightly (e.g. "Software Engineer Intern - Summer 2027" vs "SDE Intern - Summer 2027").
* We parse the exact ATS requisition ID from the URL (e.g. Greenhouse `token` or `gh_jid`, Lever UUID, Ashby UUID, Workday requisition code).
* We then generate a SHA-256 hash from `${companySlug}|${normalizedTitle}|${requisitionId}`.
* In PostgreSQL, this `dedupHash` has a `UNIQUE` constraint index. If an existing hash is encountered during ingestion, Prisma performs an `upsert`, updating the timestamps rather than creating duplicate rows.

Q5: What database indexes did you create and why?

Answer:
* `@@index([status])`: 95% of user queries filter by `WHERE status = 'OPEN'`. Indexing `status` avoids a sequential scan across thousands of closed/historical records.
* `@@index([roleCategory])`: Enables fast filtering when users select categories like `AI_ML` or `SDE`.
* `@@index([companyId])`: Speeds up foreign key joins when rendering company profile pages.
* `@@index([dedupHash])`: Enables O(log N) lookups during batch upserts in the ingestion pipeline.

Q6: How do you handle database connection pooling in a serverless environment like Neon?

Answer:
* Serverless backends or rapid scraper spikes can quickly exhaust PostgreSQL's maximum connection limit.
* We utilize Neon's built-in **PgBouncer connection pooler** (port 5432 with `-pooler` endpoint), which maintains a pool of persistent connections and multiplexes thousands of transient client requests safely without running out of database connections.

Full-Stack & Architecture Decisions

Q7: Why did you separate the frontend (Next.js) and backend (Express) instead of using Next.js API routes for everything?

Answer:
* **Separation of Concerns**: The Express backend acts as an independent microservice responsible for heavy background jobs, scheduled cron scrapers, token authentication, and business logic.
* **Long-Running Background Processes**: Next.js API routes on serverless hosts (like Vercel) have strict execution timeouts (10–15 seconds), making multi-minute scraping cycles impossible. Having a dedicated Express server with Node.js background timers (`scheduler.js`) provides persistent, uninterrupted background execution.
* **Flexibility**: The Express REST API can serve multiple client interfaces simultaneously (web app, mobile app, Telegram alerts bot).

Q8: How does user authentication work in your application?

Answer:
* We support dual-mode authentication:
1. **Password-based**: Passwords are salted with 10 rounds of `bcryptjs` and stored securely.
2. **Passwordless OTP**: Users receive a cryptographically random 6-digit OTP via email (`crypto.randomInt(100000, 999999)`), valid for 10 minutes, stored in an `OtpVerification` table.
* Once verified, the server issues a cryptographically signed **JSON Web Token (JWT)** containing the user's ID and role, signed with an HMAC SHA-256 secret key.
* The frontend attaches this token in the `Authorization: Bearer <token>` header for protected routes (e.g. bookmarks).

Q9: How do you handle search with typo tolerance and high performance?

Answer:
* In our Prisma query builder, the search parameter (`q`) executes an `insensitive` multi-field match across `title`, `description`, company `name`, and the `skills` array.
* On the frontend, the `SearchBar` component uses a controlled form that triggers on Enter key or button submission rather than firing an API request on every single keystroke, drastically reducing unnecessary server load and preventing race conditions.

Real-World Engineering Bugs & How You Solved Them

Q10: What was the most challenging bug you encountered in this project?

Answer:

> *"One of the toughest challenges was handling Greenhouse Embed URLs. Initially, our scraper collected URLs in the format `https://job-boards.greenhouse.io/embed/job_app?for=company&token=123`. When users clicked these, they were rendered in an isolated iframe view with broken CSS and non-functional submission buttons.*
> *To fix this, we wrote a URL canonicalization engine that transforms embed tokens into direct candidate application URLs. Furthermore, for companies like Stripe and Tower Research Capital that use custom job portal wrappers around Greenhouse, standard canonical URLs failed. We reverse-engineered their routing schemes to output direct URLs like `https://stripe.com/jobs/search?gh_jid=123` and `https://www.tower-research.com/open-positions/?gh_jid=123`, achieving 100% direct access to the application forms."*

Q11: Tell us about a concurrency or race condition bug you faced.

Answer:

> *"When the Express API started, the automated scheduler immediately kicked off a live sync cycle. However, if an administrator or script simultaneously triggered another sync, both processes tried to upsert the same records concurrently, causing PostgreSQL* `P2002: Unique constraint failed on dedupHash` *errors.*
> *We solved this by implementing an atomic concurrency lock (*`isRunning` *mutex flag) in* `scheduler.js`*. If a sync is already active, any subsequent requests are gracefully skipped. Additionally, in our upsert logic, we ensured that pre-existing colliding records are detected and merged within a transaction."*

---

4. Key Engineering Metrics to Highlight in Interviews

Metric	Value to Quote	Why It Impresses the Interviewer
Link Health	**100% HTTP 200 Verified**	Proves end-to-end data quality obsession.
Compensation Range	**₹35,000 to ₹1,80,000 / month**	Demonstrates clear user persona targeting (high-tier CSE students).
Database Response Time	**< 35ms**	Shows knowledge of B-Tree indexing and query optimization.
Ingestion Coverage	**4 Major ATS Systems + Workday CXS**	Shows API integration depth beyond trivial web scraping.
Build Stability	**0 Errors, 9 Next.js Production Routes**	Demonstrates clean code quality and TypeScript/ESLint compliance.

---

5. Future Architectural Enhancements

If the interviewer asks: *"How would you scale this to 100,000 active users and 10,000 companies?"*

1. **Distributed Task Queue with Redis & BullMQ**:
   Replace the in-process Node.js scheduler with a distributed worker pool using BullMQ and Redis. Different worker nodes can scrape different ATS platforms in parallel without burdening the API server.
2. **Elasticsearch / Meilisearch Full-Text Engine**:
   Offload complex keyword queries, fuzzy searching, and skill-matching from PostgreSQL to Meilisearch or Elasticsearch for sub-5ms search with typo tolerance.
3. **Kafka Event Streaming for Alert Notifications**:
   Publish newly discovered internships to an Apache Kafka topic. Downstream consumer microservices can handle sending instant email digests, Telegram bot notifications, and Discord webhook announcements.
4. **AI-Powered Resume-Role Matching**:
   Use embeddings (e.g. OpenAI `text-embedding-3-small` or HuggingFace models) stored in `pgvector` to

compute cosine similarity between a student's resume and job requirements, giving students an instant match score (e.g. 92% match).